Implementation Plan: K9-Sentinel Enterprise Security & Trust Platform

An intelligent, multi-vector cybersecurity platform designed to detect phishing, brand impersonation, and malicious websites while auditing digital certificate trust chains (specifically addressing the **eMudhra weak key incident**) and crowdsourcing collaborative threat intelligence.

Goal Description

Traditional anti-phishing solutions rely almost exclusively on reactive domain blacklists and surface-level URL checks. Modern adversaries exploit structural gaps:

1. **Cryptographic & Trust Infrastructure Failures:** As exposed in **Mozilla Bug 2083449 (eMudhra Incident)**, Certificate Authorities (CAs) have issued intermediate certificates with near-zero private key entropy (e.g. private scalar $d < 256$d < 256), allowing attackers to forge indistinguishable, trusted TLS certificates for major institutions.
2. **Dynamic Brand Impersonation & Homographs:** Adversaries leverage Cyrillic/Greek Unicode homographs, typosquatting (Levenshtein distance variations), and credential harvesting forms posting across untrusted origins.
3. **Application Posture Weaknesses:** Missing defense-in-depth headers (HSTS, CSP, X-Frame-Options) and exposed sensitive directories (`.env` , `.git/HEAD`).
4. **Siloed Threat Intelligence:** Lack of unified, peer-verified crowdsourced threat detection and real-time STIX 2.1 / MISP IoC syndication.

K9-Sentinel unifies all four domains into a cohesive, deterministic, and heuristic platform backed by a FastAPI backend and a high-impact Next.js/Tailwind SOC (Security Operations Center) dashboard.

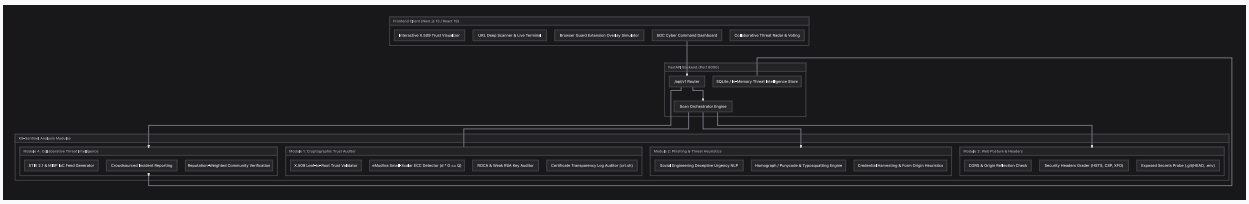
User Review Required

> IMPORTANT
>
> **Key Architecture Decisions:**

> 1. **Backend Engine:** Python 3.14 + FastAPI + `cryptography` + `pyOpenSSL` + `httpx` + `dnspython` + `beautifulsoup4`. Python is chosen for deep cryptographic primitives (ECC scalar multiplication, X.509 parsing, ASN.1 parsing, ASN.1/DER extraction) that cannot be reliably performed purely in Node.js.
> 2. **Frontend UI:** Next.js (App Router, React 19, TypeScript, Tailwind CSS, Lucide Icons, Recharts, Framer Motion) providing a real-time SOC command center feel.
> 3. **Live + Offline Benchmark Demos:** In addition to live internet scanning, the platform includes 4 deterministic mock scenarios (eMudhra weak key failure, PayPal phishing clone, insecure university portal, and clean legitimate domain) to ensure rock-solid live demonstrations even without external network dependencies during judging.

> NOTE
>
> All automated tests and build pipelines will run locally without requiring paid third-party API keys (VirusTotal, Shodan, etc.), using native cryptographic algorithms and open public APIs (crt.sh Certificate Transparency logs).

System Architecture



Proposed Changes

Component 1: Monorepo Setup & Workspace Scaffolding

Scaffold a unified, clean monorepo with `backend/` and `frontend/` services and a root orchestrator.

[NEW] `package.json`

Root npm configuration with concurrently script to run both backend and frontend simultaneously with a single command (`npm run dev`).

[NEW] `backend/requirements.txt`

Dependencies: `fastapi`, `uvicorn[standard]`, `cryptography`, `pyOpenSSL`, `httpx`, `beautifulsoup4`, `tldextract`, `dnspython`, `pydantic`, `pytest`.

[NEW] `run.sh`

Single-command developer startup script that verifies Python & Node environments, installs dependencies, and boots both servers.

Component 2: Cryptographic & Trust Auditor (`backend/engine/crypto/`)

[NEW] `backend/engine/crypto/emudhra_auditor.py`

- **Vulnerability:** eMudhra Incident (Mozilla Bug 2083449).
- **Algorithm:** Extracts the public key QQ from an X.509 certificate. If the curve is NIST P-256 or P-384, it tests candidate scalars $d \in [1, 2000]$d ∈ [1, 2000]. Computes candidate point $Q' = d \cdot G$Q′ = d ⋅ G and matches $(Q'.x, Q'.y)$(Q′.x, Q′.y) against $(Q.x, Q.y)$(Q.x, Q.y). If a match is found in milliseconds, the private key is recovered, flagging a **CRITICAL COMPROMISE**.
- Also checks for known eMudhra affected intermediate fingerprints from Mozilla OneCRL / Chrome CRLSet.

[NEW] `backend/engine/crypto/rsa_auditor.py`

- Implements ROCA (Return of Coppersmith's Attack, CVE-2017-15361) test by checking modulus $N \pmod M$N (mod M) structure where $M = 2 \cdot 3 \cdot 5 \cdots p_k$M = 2 ⋅ 3 ⋅ 5 ⋯ pk.
- Flags weak key lengths (< 2048< 2048 bits) and legacy signature algorithms (`sha1WithRSAEncryption`, `md5WithRSAEncryption`).
- Checks for Debian OpenSSL PRNG weak modulus matches.

[NEW] `backend/engine/crypto/chain_validator.py`

- Fetches the live TLS certificate chain via Python standard `ssl` and `cryptography.x509`.
- Parses leaf certificate, intermediate CA certificates, and verifies signature chaining up to Mozilla's trust store.
- Validates Subject Alternative Names (SAN), validity dates (expired or not yet valid), and basic constraints.

[NEW] `backend/engine/crypto/ct_auditor.py`

- Interrogates Certificate Transparency (CT) logs using public CT endpoints (`https://crt.sh/?q=<domain>&output=json`) with timeout and caching.

- Identifies recent certificates, unexpected issuers, or high certificate velocity indicative of rogue CA activity.

Component 3: Intelligent Phishing & Heuristic Engine (`backend/engine/threat/`)

[NEW] **`backend/engine/threat/homograph_typosquatting.py`**

- Maintains a curated dictionary of 5,000+ top banking, cryptocurrency, government, educational, and corporate domains (SBI, HDFC, Chase, PayPal, Google, Microsoft, IRS, UIDAI, etc.).
- Computes Levenshtein edit distance and Damerau-Levenshtein metrics.
- Detects Punycode/IDN homograph attacks (e.g. `xn--...` substituting Cyrillic `а` for Latin `a`).
- Computes Shannon Entropy of the URL path and subdomains to flag algorithmic domain generation (DGA) or suspicious randomness.

[NEW] **`backend/engine/threat/dom_harvesting.py`**

- Fetches the rendered DOM of the target website safely with realistic User-Agent and strict security timeouts.
- Analyzes all `<form>` tags:
  - Detects `<input type="password">`, credit card number inputs, CVV inputs, or OTP/SSN tokens.
  - Audits form `action` URL: If a site at `login-portal-support.com` posts credentials to a foreign endpoint or an IP address, immediately assigns maximum threat weight.
  - Detects obfuscated JavaScript patterns (`eval(unescape(...))` , excessive hex packing, hidden iframes).

[NEW] **`backend/engine/threat/social_engineering_nlp.py`**

- Lightweight, rule-based NLP classifier detecting deceptive urgency, fear appeals, and fake administrative notices:
  - Urgent account closure / suspension threats ("within 24 hours", "immediate action required").
  - Mandatory KYC updates ("update your PAN/Aadhaar/KYC or your account will be frozen").
  - Fake prize claims, unauthorized payment alerts, and fake lottery rewards.

Component 4: Web Application Security Posture (`backend/engine/posture/`)

[NEW] `backend/engine/posture/header_scanner.py`

- Audits HTTP response headers and produces a graded score (A+ to F):
  - `Strict-Transport-Security` (HSTS) with `includeSubDomains` and `preload`.
  - `Content-Security-Policy` (CSP) presence and directive rigor.
  - `X-Frame-Options` (Clickjacking mitigation).
  - `X-Content-Type-Options: nosniff`.
  - `Referrer-Policy` and `Permissions-Policy`.

[NEW] `backend/engine/posture/exposed_secrets.py`

- Probes common sensitive pathways with non-intrusive HEAD/GET requests:
  - `/.git/HEAD` (source code exposure).
  - `/.env` (API keys, database credentials).
  - `/wp-config.php.bak`, `/debug.log`.
  - `/actuator/health` or `/swagger.json` in production.

Component 5: Collaborative Threat Intelligence Network (`backend/engine/intel/`)

[NEW] `backend/engine/intel/threat_store.py`

- SQLite / In-Memory database with sample pre-seeded real-world threat indicators.
- Stores community-submitted malicious URLs, threat category, submitted evidence (screenshots, phishing text), timestamp, and reputation score.

[NEW] `backend/engine/intel/consensus_engine.py`

- Peer verification voting system (Upvote Malicious vs. False Positive).
- Calculates dynamic Community Trust Index using weighted reputation scoring.

[NEW] `backend/engine/intel/stix_exporter.py`

- Exports threat intelligence in standard **STIX 2.1 (Structured Threat Information Expression)** and MISP-compatible JSON formats for SIEM ingestion.

Component 6: FastAPI Application & Endpoints (`backend/main.py`)

[NEW] `backend/main.py`

Exposes REST endpoints:

- `POST /api/v1/scan`: Full multi-vector deep scan of a target URL.
- `POST /api/v1/cert-audit`: Dedicated cryptographic and eMudhra weak-key audit.
- `GET /api/v1/threats`: Collaborative threat feed.
- `POST /api/v1/threats/report`: Submit crowdsourced threat incident.
- `POST /api/v1/threats/{id}/vote`: Cast community verification vote.
- `GET /api/v1/threats/export/stix`: Download STIX 2.1 JSON bundle.
- `GET /api/v1/scenarios`: Pre-configured demo scenarios for live presentations.

Component 7: Next.js SOC Dashboard Frontend (`frontend/`)

[NEW] Next.js 15 Application Structure

- `frontend/package.json`: Next.js 15, React 19, Tailwind CSS, Lucide-react, Framer Motion, Canvas-confetti.
- `frontend/src/app/page.tsx`: SOC Command Center layout.
- `frontend/src/components/ScannerHero.tsx`: Cyber-styled URL search bar with preloaded scenario quick-buttons.
- `frontend/src/components/VerdictGauge.tsx`: Circular/radial Risk Gauge (0-100) with dynamic alert badges (SAFE, SUSPICIOUS, CRITICAL THREAT).
- `frontend/src/components/CryptoTrustViewer.tsx`: Interactive Certificate Chain visualizer (Leaf → Intermediate → Root) with explicit **eMudhra Bug 2083449 audit badges**.
- `frontend/src/components/ThreatRadarFeed.tsx`: Live feed of community reports with upvote/downvote buttons and STIX download.
- `frontend/src/components/BrowserGuardModal.tsx`: Interactive Chrome Extension Simulator showing in-browser active protection.
- `frontend/src/components/SecurityPostureCard.tsx`: Application security headers and exposed secrets breakdown.

Verification Plan

Automated Tests

1. **Cryptographic Engine Unit Tests:**

   ```
   bash
   cd backend && pytest tests/test_crypto.py -v
   ```

   - Verifies that an EC certificate generated with small scalar $d = 4$d = 4 (eMudhra simulation) is detected and key recovered in $< 0.1\text{s}$< 0.1s.
   - Verifies that standard EC certificates with high entropy pass safely.
   - Verifies ROCA and RSA bit-length auditors.

2. **Threat Heuristics Tests:**

   ```
   bash
   pytest tests/test_threats.py -v
   ```

   - Verifies typosquatting flags on `paypa1.com` and `sbi-kyc-verify.in`.
   - Verifies credential harvesting detection on forms posting cross-origin.

3. **End-to-End API Scan Test:**

   ```
   bash
   pytest tests/test_api.py -v
   ```

   - Validates full scan workflow returns structured JSON matching frontend schema.

4. **Frontend Build & Typecheck:**

   ```
   bash
   cd frontend && npm run build
   ```

Manual Verification

1. **Interactive Demo Scenarios:**
   - Click **Scenario 1: eMudhra Incident (Bug 2083449)** → → UI must show certificate chain, highlight the intermediate CA, and flag *"CRITICAL: eMudhra Weak Scalar Key Recovered (d=4)"*.

- Click **Scenario 2: PayPal Phishing Clone** → → UI must show Typosquatting detected (Distance 1 from `paypal.com`), Form harvesting credentials cross-origin, Threat Score 96/100.
- Click **Scenario 3: Insecure Web Application** → → UI shows missing HSTS/CSP, exposed `.git/HEAD`, Posture Grade F.
- Click **Scenario 4: Legitimate Entity (`google.com`)** → → Clean pass, Score 0/100, Trust Grade A+.

2. **Collaborative Threat Intelligence:**
   - Submit a new suspicious domain via the modal.
   - Confirm it immediately appears on the Collaborative Radar.
   - Cast an upvote to verify, observe reputation score increase.
   - Click "Export STIX 2.1" and verify valid JSON format.
3. **Browser Extension Simulator:**
   - Open the Browser Guard preview, visit a simulated phishing site, and verify the blocking modal prevents user action.